

CI/CD FOR DATA ENGINEERING

Detailed architecture, testing, deployment, rollback and hands-on implementation

Training objective: Move beyond the definition of CI/CD and understand what actually happens from a developer changing a pipeline through production deployment. Examples use Python/PySpark, Databricks, ADF and Airflow patterns.

1. The Production Problem CI/CD Solves

A data engineer rarely changes only a single isolated script. A production change can affect transformations, schemas, job definitions, libraries, infrastructure, permissions and downstream consumers. Manual deployment makes those changes difficult to reproduce and audit.

```
Developer laptop
|
| manually edits notebook
v
DEV workspace
|
| copy/paste
v
PROD workspace

Problems:
- DEV and PROD code drift
- forgotten configuration changes
- no repeatable test gate
- credentials copied manually
- difficult to identify exact deployed version
- rollback depends on memory
```

2. End-to-End CI/CD Architecture

The pipeline should be understood as two connected systems: the software delivery pipeline and the data-processing pipeline. CI/CD deploys the code and platform definitions; the deployed workload then processes business data.

```
SOFTWARE DELIVERY FLOW

Developer
|
feature branch
v
Git Repository
|
Pull Request
v
CI Pipeline
lint -> unit tests -> transformation tests -> security checks
|
Merge
v
Build / Version
```

```

|
CD Pipeline
|
DEV -> QA -> PROD
|
v
DEPLOYED DATA PIPELINE

```

Sources -> Ingestion -> Transform -> Curated Tables -> BI/ML

3. CI, Continuous Delivery and Continuous Deployment

Continuous Integration validates changes frequently. Continuous Delivery keeps a validated release ready for controlled deployment. Continuous Deployment removes the manual production release step when an organization has enough automated confidence.

Practice	Primary purpose	Typical trigger
CI	Validate change before merge	Push / Pull Request
Continuous Delivery	Prepare/promote a validated release	Merge/release + controlled approval
Continuous Deployment	Automatically release to production	Successful automated gates

4. Source Control and Branching

Git gives every change an identity and history. The repository becomes the source of truth for deployable code rather than a notebook or pipeline edited directly in production.

```

main
|
+-- commit A ---- commit B ----- merge PR ----- commit D
                        \          /
                        feature/orders-v2
                          commit C1
                          commit C2

```

5. Pull Requests and Code Review

A Pull Request is a governance point for engineering changes. Reviewers should not only check syntax; they should examine business logic, performance, backward compatibility, data-quality impact, security and operational behavior.

Review area	Question
Correctness	Does transformation implement the requirement?
Data impact	Can it duplicate/drop/change records unexpectedly?
Performance	New shuffle, full scan, skew or inefficient API call?
Compatibility	Will downstream schema/contracts break?
Security	Any secret, PII exposure or privilege expansion?
Operations	Retries, logging, idempotency and failure handling?

6. Repository Design

A repository should separate transformation code, tests, deployment definitions and environment configuration. This improves reuse and prevents the common anti-pattern of maintaining a different copy of the same pipeline for every environment.

```
orders-platform/  
  src/  
    orders/  
      transform.py  
      validation.py  
  tests/  
    unit/  
    integration/  
  sql/  
  databricks.yml  
  resources/  
    orders_job.yml  
  infra/  
    terraform/  
  config/  
    dev.yml  
    qa.yml  
    prod.yml  
  .github/workflows/  
    ci.yml  
    deploy.yml  
  requirements.txt
```

7. CI Pipeline Stages

A CI pipeline is a sequence of automated gates. Fast checks should normally run first so obvious failures are detected before expensive integration tests.

```
PR opened  
|  
v  
Checkout  
|  
v  
Dependency install  
|  
+--> formatting/lint  
+--> unit tests  
+--> PySpark/SQL tests  
+--> schema/contract validation  
+--> security scan  
+--> deployment-definition validation  
|  
v  
PASS -> merge eligible  
FAIL -> developer fixes and pushes again
```

8. Unit Testing Data Transformations

Unit tests validate a small deterministic behavior. Good transformation code separates business logic from orchestration so it can be tested without launching an entire production workflow.

```
def calculate_revenue(quantity, unit_price):  
    if quantity < 0 or unit_price < 0:  
        raise ValueError("Invalid order values")  
    return quantity * unit_price
```

```
def test_calculate_revenue():
    assert calculate_revenue(2, 100.0) == 200.0
```

9. PySpark Transformation Testing

For PySpark, create a tiny DataFrame, execute the transformation and compare the resulting rows/schema with the expected result. The goal is to test logic, not production volume.

```
Input
order_id  qty  price  status
1         2   100   Completed
2         1   500   Cancelled
```

```
Transformation
- retain Completed
- revenue = qty * price
```

```
Expected
order_id  revenue
1         200
```

10. Data Quality Tests in CI vs Runtime

Not every data-quality rule belongs in CI. CI can test logic and contracts using controlled test data. Runtime quality checks validate the actual production data arriving today.

CI test	Runtime production check
Function rejects null business key in test fixture	Today's source contains no null order_id
Schema contract expects amount DECIMAL	Production table schema still conforms
Deduplication logic chooses latest record	Duplicate rate today is below threshold

11. Integration and Smoke Testing

Integration tests validate boundaries between components, such as reading a test bucket, authenticating to a development service or writing a temporary table. Smoke tests run after deployment and answer a simpler question: did the deployed release start and perform a basic expected action?

12. Build Artifacts and Versioning

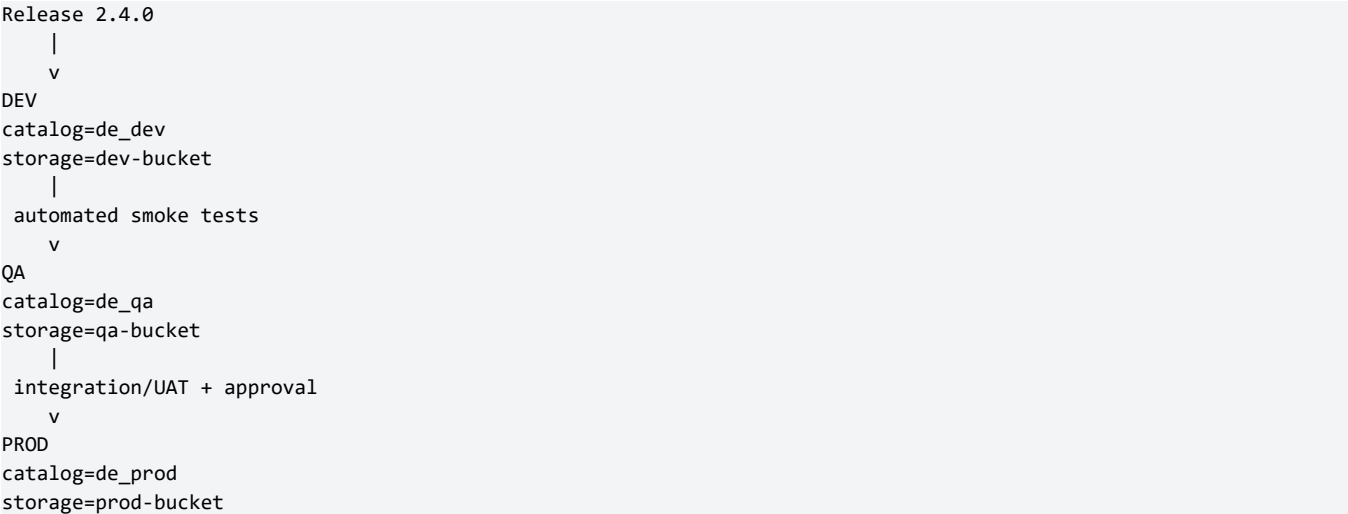
A release should be identifiable. Depending on the platform, the artifact may be a Python wheel, container image, Databricks bundle configuration, Terraform version or another immutable package. The exact version deployed to production should be traceable to a Git commit.

```
Git commit: abc123
|
v
Build artifact
orders_pipeline-2.4.0.whl
|
+--> DEV
+--> QA
+--> PROD
```

Deployment record:
PROD = version 2.4.0 = commit abc123

13. CD and Environment Promotion

Promotion means moving a tested release through environments. Ideally, code is not rewritten between environments; configuration changes while the release remains the same.



14. Configuration Management

Environment configuration should be externalized. A production catalog name, storage account or endpoint should not require changing transformation logic.

```
# transformation code
catalog = config["catalog"]
target = f"{catalog}.gold.orders"

# dev.yml
catalog: de_dev

# prod.yml
catalog: de_prod
```

15. Secrets and Deployment Identities

Passwords, tokens and client secrets must not be committed to Git. CI/CD should use secret stores or workload identities/service principals with least privilege. Production deployment identity should be auditable and separate from a developer's personal login.

Good pattern	Bad pattern
OIDC/workload identity or secret manager	Cloud password in YAML
Service principal with scoped deployment rights	Developer personal token used by production pipeline
Secret reference resolved at runtime	Secret committed to repository

16. Approval Gates and Separation of Duties

A production gate is appropriate when risk requires a human decision after automated validation. In regulated environments, the person who wrote the code may not be the only person authorized to approve production deployment.

```
Automated gates pass
  |
  v
PROD approval
  Reviewer sees:
  - release version
  - test results
  - changed resources
  - deployment plan
  |
  approve / reject
  |
  v
production deployment
```

17. Databricks CI/CD Architecture

A Databricks deployment can version Python/SQL code together with job or pipeline definitions. Databricks Asset Bundles provide a current project-based deployment mechanism for describing resources and targets.

```
GitHub
  |
PR + CI
  |
  +-- pytest
  +-- lint
  +-- bundle validate
  |
merge main
  |
CD identity
  |
  +--> bundle deploy -t dev
  +--> validation run
  +--> bundle deploy -t prod
  |
Databricks
  |
Workflow / Job / Pipeline
  |
Bronze -> Silver -> Gold
```

18. ADF CI/CD Architecture

ADF development artifacts include pipelines, datasets, linked services, triggers and parameters. The core deployment principle is to version definitions, validate them, parameterize environment-specific values and deploy through controlled environments.

```
ADF DEV + source control
  |
  v
Pipeline definitions
  |
  Build
  |
validation / templates
  |
  v
```

```

Release pipeline
  |           |
  v           v
ADF QA      ADF PROD
  |           |
Key Vault / environment parameters

```

19. Airflow CI/CD Architecture

Airflow DAGs are Python software. CI should catch import errors and test reusable task logic before DAGs reach the scheduler. CD then places the approved DAG/package in the environment's deployment location.

```

DAG PR
|
CI
+-- Python import test
+-- DAG parse test
+-- unit tests for task functions
+-- lint
|
merge
|
CD
|
Airflow DEV -> Airflow PROD

```

20. Infrastructure as Code

Infrastructure as Code extends version control to platform resources. Terraform, Bicep/ARM, CloudFormation or equivalent tools can make storage, networking, identities and job resources reproducible.

```

Pull Request
|
Terraform validate
|
Terraform plan
|
review
|
merge / approval
|
Terraform apply
|
Cloud infrastructure

```

21. Database and Data Contract Changes

Data systems need compatibility planning because consumers can break even when deployment succeeds technically. Renaming or dropping a column should be treated as a contract change, not merely a code edit.

Change	Risk	Deployment strategy
Add nullable column	Low/medium	Producer first, consumer adoption later
Rename column	High	Dual support/versioned contract during migration
Drop column	High	Lineage/impact analysis; migrate consumers before removal
Change type STRING -> INT	High	Profile data, backfill, compatibility test

22. Rollback, Roll Forward and Data Recovery

Software rollback restores code. Data recovery restores data state. These are not the same. If release 2.4 writes incorrect revenue for three hours, deploying 2.3 stops future bad writes but does not repair already corrupted rows.

```
Release 2.4 deployed
|
+--> code crashes
|     -> rollback artifact
|
+--> writes bad data
|     -> rollback code
|     -> identify affected interval
|     -> restore/reprocess data
|
+--> destructive schema migration
|     -> recovery/migration plan
```

23. Observability After Deployment

A release is not complete when deployment reports success. Monitor job success, duration, data volume, freshness, quality failures, infrastructure errors and downstream SLA impact.

Signal	Example
Job health	success/failure/retry
Performance	duration changed 10 min -> 90 min
Volume	expected 5M rows, received 50K
Freshness	gold table older than SLA
Quality	null/duplicate threshold breached
Cost	compute usage unexpectedly increased

24. End-to-End Example

Consider a PySpark orders pipeline where a developer changes the Completed-order revenue calculation. The full engineering lifecycle should make the change traceable from requirement to production behavior.

```
Ticket
|
feature branch
|
code + tests
|
Pull Request
|
CI
unit + PySpark + schema + lint
|
review
|
merge
|
build release 3.1.0
|
```

```

deploy DEV
|
smoke + sample data validation
|
deploy QA
|
integration tests
|
production approval
|
deploy PROD
|
monitor freshness/quality/runtime
|
close release

```

25. Detailed GitHub Actions Lab

1. Create a repository named `orders-cicd-lab`.
2. Create `src/orders.py` and implement a small `calculate\_revenue` or order-cleaning function.
3. Create `tests/test\_orders.py` with positive, invalid and boundary cases.
4. Create `requirements.txt` containing pytest.
5. Create `.github/workflows/ci.yml`.
6. Configure the workflow for Pull Requests and pushes to main.
7. Checkout the repository, install Python, install requirements and run pytest.
8. Create a feature branch and intentionally introduce a failing expectation.
9. Push and open a Pull Request. Observe the failed workflow.
10. Fix the code/test and push to the same branch. Observe automatic rerun.
11. Merge after the workflow passes.
12. Add a second job named `deploy-dev` that only runs after tests on main. For training, make it print the release/environment rather than deploying a real cloud resource.
13. Explain where credentials, production approval and a real Databricks/ADF deployment command would be inserted.

```

name: orders-ci

on:
  pull_request:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install -r requirements.txt
      - run: pytest -v

  deploy-dev:
    if: github.ref == 'refs/heads/main'
    needs: test
    runs-on: ubuntu-latest
    steps:
      - run: echo "Deploy validated release to DEV"

```

26. CI/CD Troubleshooting Scenarios

Symptom	Likely investigation
Works locally, CI fails	Python/library version, missing env var, case-sensitive path, undeclared dependency
DEV deploy works, PROD fails	Permissions, secret scope, network, environment config, missing production resource
Deployment succeeds but job fails	Runtime input/schema/data issue, missing library, service access
Pipeline suddenly slower after release	Spark plan/shuffle/skew, changed partitioning, data growth, compute config
Rollback code succeeds but dashboard still wrong	Bad data already persisted; repair/reprocess required

27. Review and Interview Questions

14. Walk through a complete CI/CD pipeline for a PySpark job.
15. What is the difference between CI, continuous delivery and continuous deployment?
16. Why should business logic be separated from orchestration?
17. Which tests belong in CI and which belong at runtime?
18. How do you promote the same code across DEV/QA/PROD?
19. How do service principals/workload identities fit CI/CD?
20. What is an immutable artifact?
21. How do you deploy Databricks jobs through CI/CD?
22. How would you test an Airflow DAG?
23. Why is data rollback different from code rollback?
24. How do you manage a breaking schema change?
25. What should be monitored immediately after deployment?